

TABLE OF CONTENTS

Flutter App Architecture: The Application Layer	1
Shopping Cart: UI Overview	3
Multiple widgets, shared logic?	4
Adding items as a guest or logged-in user	4
Shopping Cart: Full Requirements	5
The Application Layer	5
Shopping Cart Implementation	6
The Cart Data Model	7
The Auth and Cart Repositories	8
The CartService class	10
Passing Ref as an argument	11
Adding an item using the CartService	11
Adding the remaining methods to the CartService	13
Implementing the ShoppingCartItemController	14
Note about controllers, services, and repositories	16
Conclusion	17
Closing notes	18
Flutter Foundations Course Now Available	18
Flutter Foundations Course	19
Want More?	19
Flutter In Production	19
Flutter & Firebase Masterclass	19
The Complete Dart Developer Guide	20
Flutter Animations Masterclass	20



Flutter App Architecture: The Application Layer

#dart

#flutter

#state-management

#app-architecture

#riverpod

**Andrea Bizzotto**

Updated Jun 10, 2022 · 15 min read

When building complex apps, we may find ourselves writing logic that:

- depends on multiple data sources or repositories
- needs to be used (shared) by more than one widget

In this case, it's tempting to put that logic inside the classes that we already have (widgets or repositories).

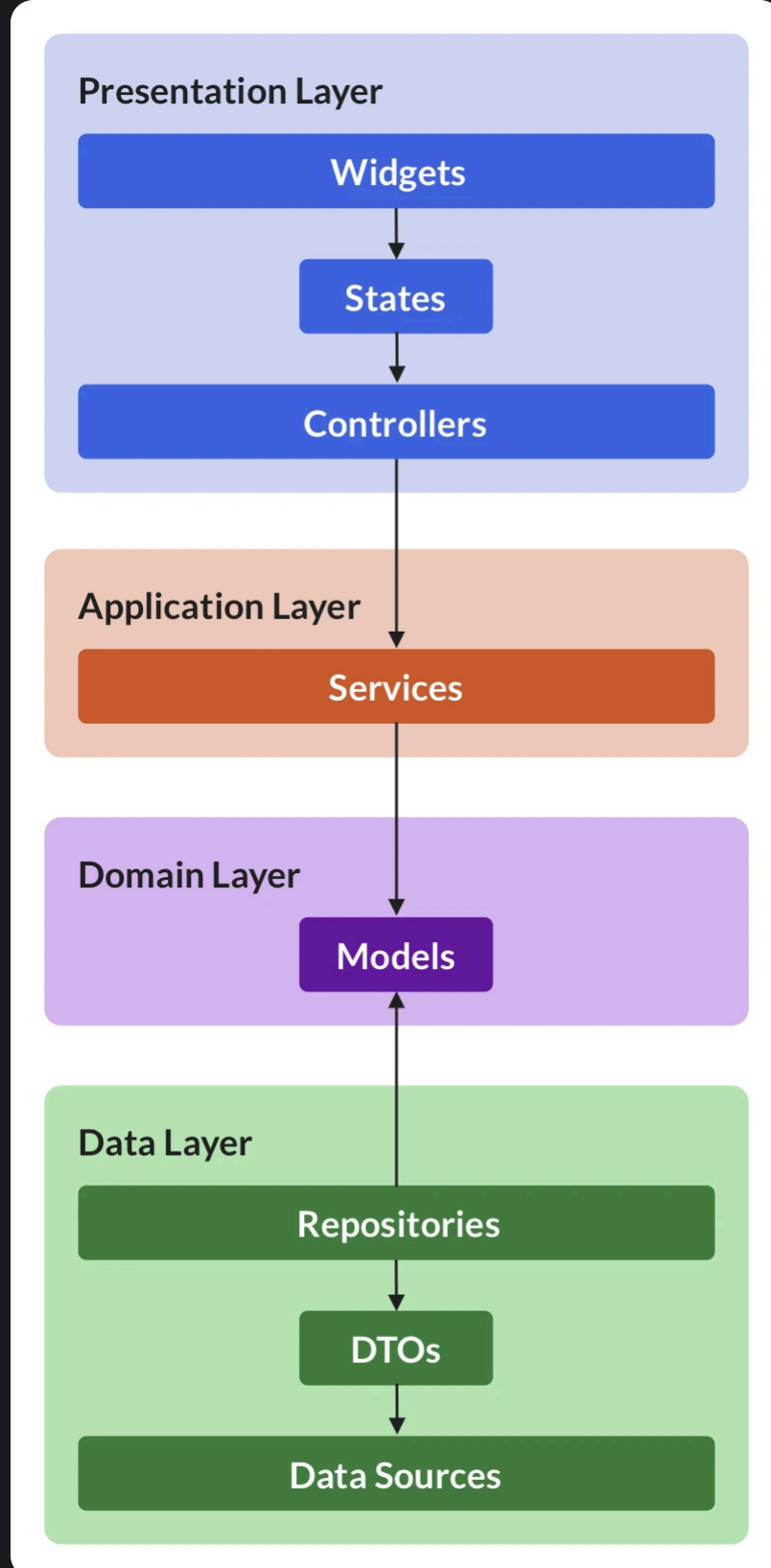
But this leads to poor separation of concerns, making our code harder to read, maintain, and test.

In fact, **separation of concerns is the #1 reason why we need a good app architecture.**

And in previous articles, I've introduced a reference app architecture that has worked very well for my projects:

- [Flutter App Architecture with Riverpod: An Introduction](#)
- [Flutter Project Structure: Feature-first or Layer-first?](#)
- [Flutter App Architecture: The Repository Pattern](#)
- [Flutter App Architecture: The Domain Model](#)
- [Flutter App Architecture: The Presentation Layer](#)

This architecture defines four separate layers with clear boundaries:



Flutter App Architecture using data, domain, application, and presentation layers. Arrows show the dependencies between layers

And in this article, we're going to focus on the **application layer** and learn how to implement a **shopping cart feature** for an eCommerce app in Flutter.

We'll start with a conceptual overview of this feature, to see how everything fits together at a high level.

And then, we'll dive into some implementation details and implement a `CartService` class that depends on multiple repositories.

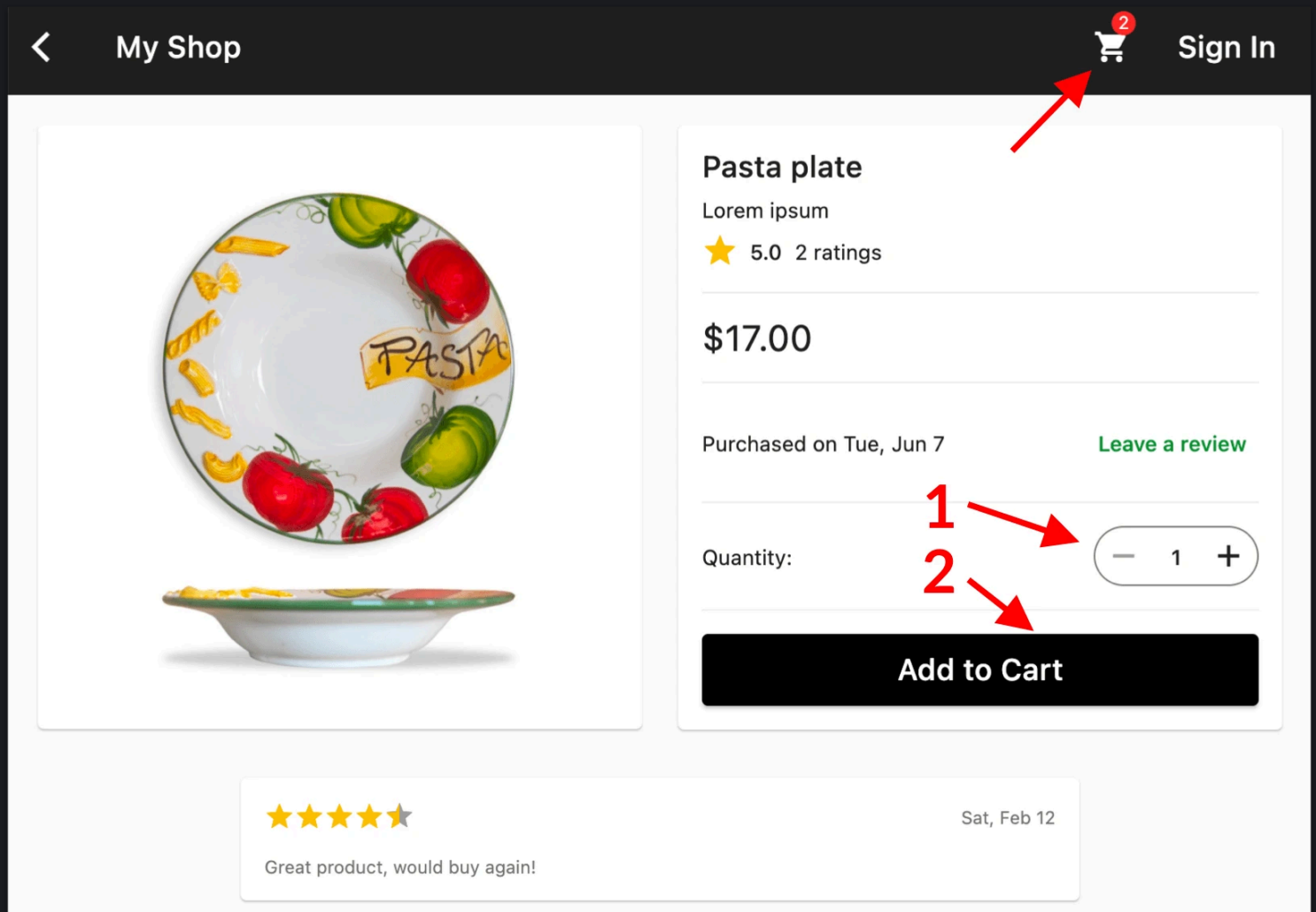
We'll also learn to easily manage multiple dependencies with Riverpod (using `Ref` inside a service class).

Ready? Let's go!

Shopping Cart: UI Overview

Let's consider some example UI we may use to implement a shopping cart feature.

At a very minimum, we need a product page:



Product page with quantity selector and "Add to Cart" button. Also, isn't that a lovely pasta plate?

This page lets us **select the desired quantity** (1) and **add the product to the cart** (2). On the top right corner, we also find a shopping cart icon with a badge telling us how many items are in the cart.

We also need a shopping cart page:

Total: \$77.00

Checkout



Mozzarella plate

\$13.00

- 2 + 

Pasta plate

\$17.00

- 3 + *Shopping cart page with options to edit quantity and remove items*

This page lets us **edit the quantity** or **remove items** from the cart.

Multiple widgets, shared logic?

As we can already see, there are **multiple** widgets (each page is a widget itself) that need access to the shopping cart data to show the correct UI.

In other words, the shopping cart items (and the logic for updating them) need to be **shared** across multiple widgets.

To make things even more interesting, let's add one more requirement.

Adding items as a guest or logged-in user

eCommerce sites such as Amazon or eBay will let you add items to the shopping cart **before** creating an account.

This way, you can freely search the product catalog as a guest and only sign in or register when you proceed to checkout.

So how can we replicate the same functionality in our example app?

One way to do it is to have **two shopping carts**:

- a **local** shopping cart used by guests
- a **remote** shopping cart used by signed-in users

With this setup, we can add an item to the correct cart with this logic:

```
if user is signed in, then
    add item to remote cart
else
    add item to local cart
```

What this means in practice is that we need **three repositories** to make things work:

- an **auth** repository, used to sign in and sign out
- a **local cart** repository, used by guest users (backed by local storage)
- a **remote cart** repository, used by authenticated users (backed by a remote database)

Shopping Cart: Full Requirements

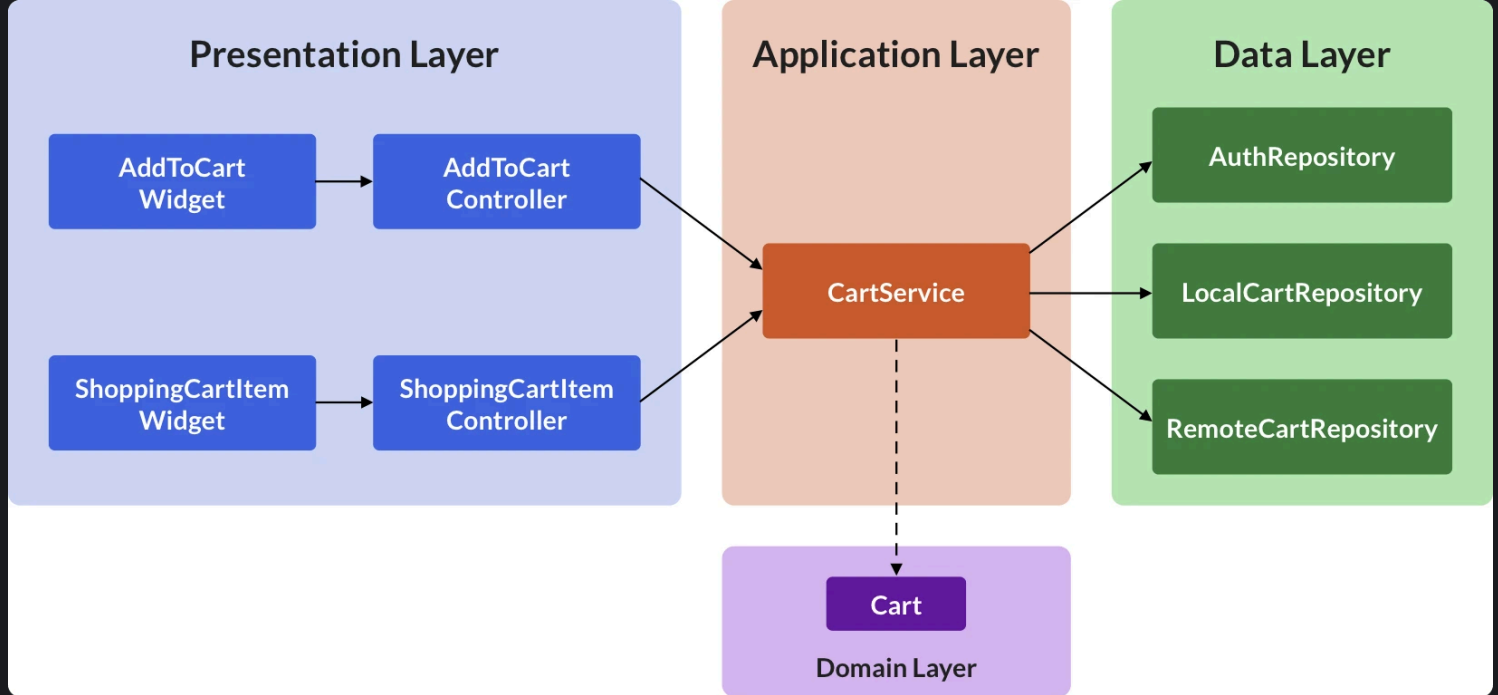
In summary, we need to be able to:

- add items to the cart as a guest or authenticated user (using different repositories)
- do so from different widgets/pages

But where should all this logic go?

The Application Layer

In this scenario, the best way to keep our code organized is to introduce an **application layer** that contains a `CartService` to hold all our logic:



Layers and components used by the shopping cart feature

As we can see, the `CartService` acts as a middle-man between the controllers (which only manage the widget state) and the repositories (which talk to different data sources).

The `CartService` is **not** concerned about:

- managing and updating the widget state (that's the job of the controller)
- data parsing and serialization (that's the job of the repositories)

All it does is to implement application-specific logic by accessing the relevant repositories as needed.

Note: other common architectures based on MVC or MVVM keep this application-specific logic (along with the data-layer code) in the model class itself. However, this can lead to models that contain too much code and are difficult to maintain. By creating repositories and services as needed, we get a much better separation of concerns.

And now that we have a clear picture of what we're trying to do, let's implement all the relevant code.

Shopping Cart Implementation

Our goal is to figure out how to implement the `CartService` class.

Since this depends on multiple data models and repositories, let's define those first.

The Cart Data Model

In essence, a shopping cart is a collection of items identified by a product ID and a quantity.

We could implement this using a **list**, a **map**, or even a **set**. What I found works best is to create a class that contains a map of values:

```
class Cart {  
  const Cart([this.items = const {}]);  
  
  /// All the items in the shopping cart, where:  
  /// - key: product ID  
  /// - value: quantity  
  final Map<ProductID, int> items;  
  /// Note: ProductID is just a String  
}
```

Since we want the `Cart` class to be **immutable** (to prevent widgets from mutating its state), we can define an **extension** with some methods that modify the current cart, and return a new `Cart` object:

```
/// Helper extension used to mutate the items in the shopping cart.  
extension MutableCart on Cart {  
  // implementations omitted for brevity  
  Cart addItem(Item item) { ... }  
  Cart setItem(Item item) { ... }  
  Cart removeItemById(ProductID productId) { ... }  
}
```

We can also define an `Item` class that holds the product ID and quantity as a single entity:

```
/// A product along with a quantity that can be added to an order/cart  
class Item {  
  const Item({  
    required this.productId,  
    required this.quantity,  
  });  
  final ProductID productId;  
  final int quantity;  
}
```

My article about [Flutter App Architecture: The Domain Model](#) offers a complete overview of these model classes. And if you want to learn how to build an entire eCommerce app, check out my [Complete Flutter Course Bundle](#).

The Auth and Cart Repositories

As discussed, we need an auth repository that we can use to check if we have a signed-in user:

```
abstract class AuthRepository {  
  /// returns null if the user is not signed in  
  AppUser? get currentUser;  
  
  /// useful to watch auth state changes in realtime  
  Stream<AppUser?> authStateChanges();  
  
  // other sign in methods  
}
```

When we're using the app as a guest, we can use a `LocalCartRepository` to get and set the cart value:

```
abstract class LocalCartRepository {  
  // get the cart value (read-once)  
  Future<Cart> fetchCart();  
  
  // get the cart value (realtime updates)  
  Stream<Cart> watchCart();  
  
  // set the cart value  
  Future<void> setCart(Cart cart);  
}
```

The `LocalCartRepository` class can be subclassed and implemented using local storage (with packages such as [Sembast](#), [ObjectBox](#), or [Isar](#)).

And if we're signed in, we can use a `RemoteCartRepository` instead:

```
abstract class RemoteCartRepository {
    // get the cart value (read-once)
    Future<Cart> fetchCart(String uid);

    // get the cart value (realtime updates)
    Stream<Cart> watchCart(String uid);

    // set the cart value
    Future<void> setCart(String uid, Cart items);
}
```

This class is very similar to the `LocalCartRepository`, with one fundamental difference: all methods take a `uid` argument since each authenticated user will have his/her own shopping cart.

If we use Riverpod, we also need to define a provider for each of these repositories:

```
final authRepositoryProvider = Provider<AuthRepository>((ref) {
    // This should be overridden in main file
    throw UnimplementedError();
});

final localCartRepositoryProvider = Provider<LocalCartRepository>((ref) {
    // This should be overridden in main file
    throw UnimplementedError();
});

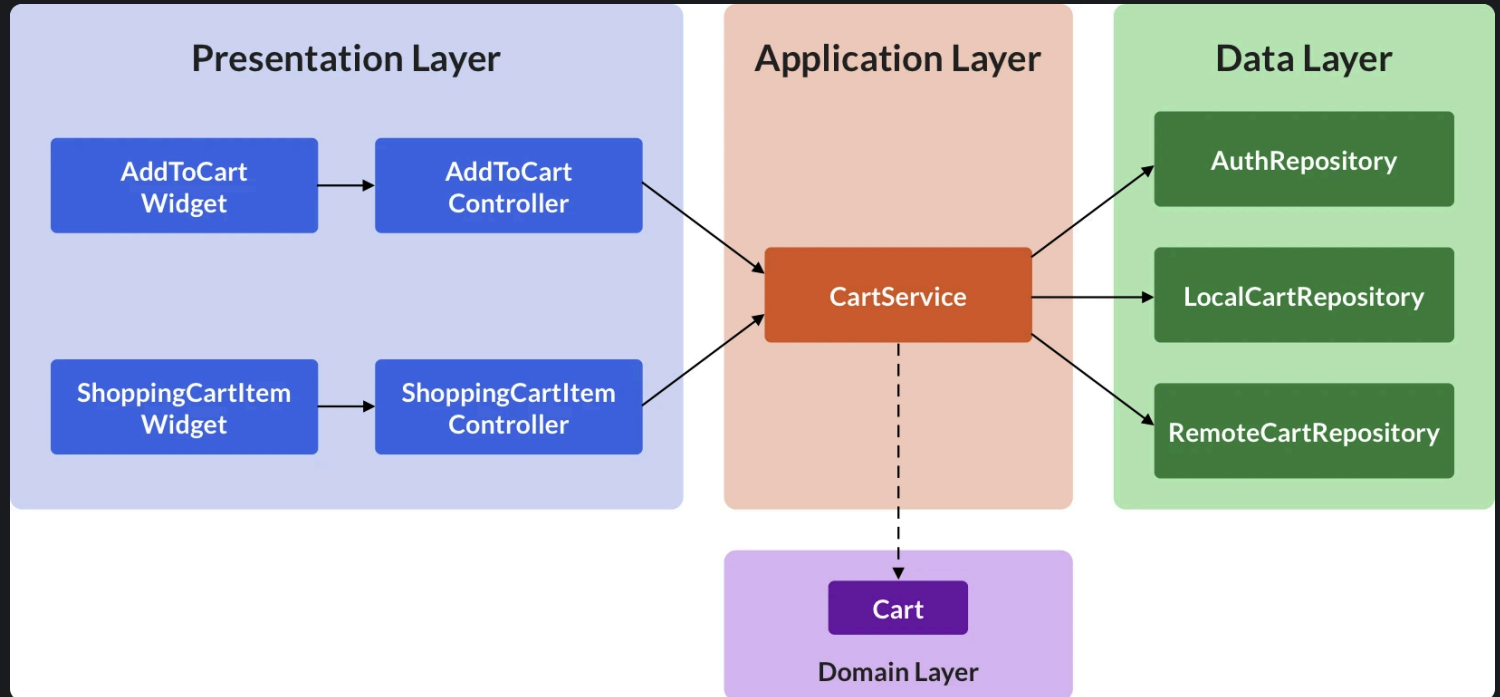
final remoteCartRepositoryProvider = Provider<RemoteCartRepository>((ref) {
    // This should be overridden in main file
    throw UnimplementedError();
});
```

Note how all these providers throw an `UnimplementedError`, since we have defined the repositories as **abstract classes**. If you only use **concrete classes**, you can instantiate and return them directly instead. For more info on this, read [this note about abstract or concrete classes](#) in my article about [Flutter App Architecture: The Repository Pattern](#).

And now that both data models and repositories are out of the way, let's focus on the service class.

The CartService class

As we can see, the `CartService` class **depends** on three separate repositories:



Layers and components used by the shopping cart feature

So we could declare them as `final` properties and pass them as constructor arguments:

```
class CartService {
  CartService({
    required this.authRepository,
    required this.localCartRepository,
    required this.remoteCartRepository,
  });
  final AuthRepository authRepository;
  final LocalCartRepository localCartRepository;
  final RemoteCartRepository remoteCartRepository;

  // TODO: implement methods using these repositories
}
```

Along the same lines, we could define the corresponding provider:

```
final cartServiceProvider = Provider<CartService>((ref) {
    return CartService(
        authRepository: ref.watch(authRepositoryProvider),
        localCartRepository: ref.watch(localCartRepositoryProvider),
        remoteCartRepository: ref.watch(remoteCartRepositoryProvider),
    );
});
```

This works and it makes all the dependencies **explicit**.

But if you don't like having so much boilerplate code, there is an alternative. 📌

Passing Ref as an argument

Rather than passing each dependency directly, we can just declare a single `Ref` property:

```
class CartService {
    CartService(this.ref);
    final Ref ref;
}
```

And when we define the provider, we just pass `ref` as an argument:

```
final cartServiceProvider = Provider<CartService>((ref) {
    return CartService(ref);
});
```

And now that we have declared the `CartService` class, let's add some methods to it.

Adding an item using the CartService

To make our life easier, we can define two private methods that we can use to **fetch** and **set** the cart value:

```
class CartService {
    CartService(this.ref);
    final Ref ref;
```

```

/// fetch the cart from the local or remote repository
/// depending on the user auth state
Future<Cart> _fetchCart() {
    final user = ref.read(authRepositoryProvider).currentUser;
    if (user != null) {
        return ref.read(remoteCartRepositoryProvider).fetchCart(user.uid);
    } else {
        return ref.read(localCartRepositoryProvider).fetchCart();
    }
}

/// save the cart to the local or remote repository
/// depending on the user auth state
Future<void> _setCart(Cart cart) async {
    final user = ref.read(authRepositoryProvider).currentUser;
    if (user != null) {
        await ref.read(remoteCartRepositoryProvider).setCart(user.uid,
cart);
    } else {
        await ref.read(localCartRepositoryProvider).setCart(cart);
    }
}
}

```

Note how we can read each repository by calling `ref.read(provider)` and invoking the methods we need on them.

By passing `Ref` as an argument, the `CartService` now depends directly on the Riverpod package and the **actual** dependencies are now **implicit**. If this is not what you want, just pass the dependencies explicitly as shown above. *Note: I'll show how to write unit tests for service classes using `Ref` in a separate article.*

Next up, we can create a public `addItem()` method that calls `_fetchCart()` and `_setCart()` under the hood:

```

class CartService {
    CartService(this.ref);
    final Ref ref;

    Future<Cart> _fetchCart() { ... }
    Future<void> _setCart(Cart cart) { ... }
}

```

```

    /// adds an item to the local or remote cart
    /// depending on the user auth state
    Future<void> addItem(Item item) async {
        // 1. fetch the cart
        final cart = await _fetchCart();
        // 2. return a copy with the updated data
        final updated = cart.addItem(item);
        // 3. set the cart with the updated data
        await _setCart(updated);
    }
}

```

What this method does is to:

1. fetch the cart (from the local or remote repository depending on the auth state)
2. make a copy and return an updated cart
3. set the cart with the updated data (using the local or remote repository depending on the auth state)

Note that the second step calls the `addItem()` method that we have previously defined in the `MutableCart` extension. The logic to mutate the `Cart` should live in the **domain** layer since it doesn't depend on any services or repositories.

Adding the remaining methods to the CartService

Just like we have defined the `addItem()` method, we can add the other methods that the controllers will use:

```

class CartService {
    ...
    /// removes an item from the local or remote cart depending on the user
    auth
    /// state
    Future<void> removeItemById(String productId) async {
        // business logic
        final cart = await _fetchCart();
        final updated = cart.removeItemById(productId);
        await _setCart(updated);
    }
}

```

```

    /// sets an item in the local or remote cart depending on the user auth
    state
    Future<void> setItem(Item item) async {
        final cart = await _fetchCart();
        final updated = cart.setItem(item);
        await _setCart(updated);
    }
}

```

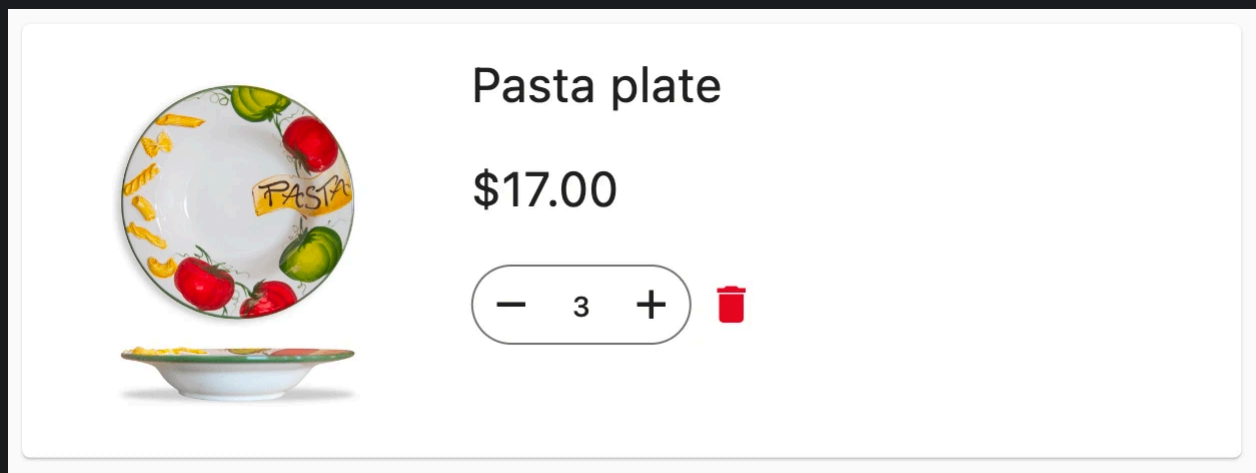
Note how the second step always **delegates** the cart update to a method in the `MutableCart` extension, which can be easily unit tested since it has no dependencies.

That's it! We've now completed the implementation of the `CartService`.

Next up, let's see how to use this inside a controller.

Implementing the ShoppingCartItemController

Let's consider how we can update or remove items that are already in the shopping cart:



A shopping cart item widget

To do this, we'll have a `ShoppingCartItem` widget and a corresponding `ShoppingCartItemController` class with `updateQuantity` and `deleteItem` methods:

```

class ShoppingCartItemController extends StateNotifier<AsyncValue<void>>
{
    ShoppingCartItemController({required this.cartService})
        : super(const AsyncData(null));
    final CartService cartService;

```

```

Future<void> updateQuantity(Item item, int quantity) async {
    // set loading state
    state = const AsyncLoading();
    // create an updated Item with the new quantity
    final updated = Item(productId: item.productId, quantity: quantity);
    // use the cartService to update the cart
    // and set the state again (data or error)
    state = await AsyncValue.guard(
        () => cartService.updateItemIfExists(updated),
    );
}

Future<void> deleteItem(Item item) async {
    state = const AsyncLoading();
    state = await AsyncValue.guard(
        () => cartService.removeItemById(item.productId),
    );
}
}

```

The methods in this class have two jobs:

- update the widget state
- call the corresponding `CartService` methods to update the cart

Note how each method has only a few lines of code. This is by design because the `CartService` holds all the complex logic, and this can be reused by other controllers too!

To wrap up, let's define the provider for this controller:

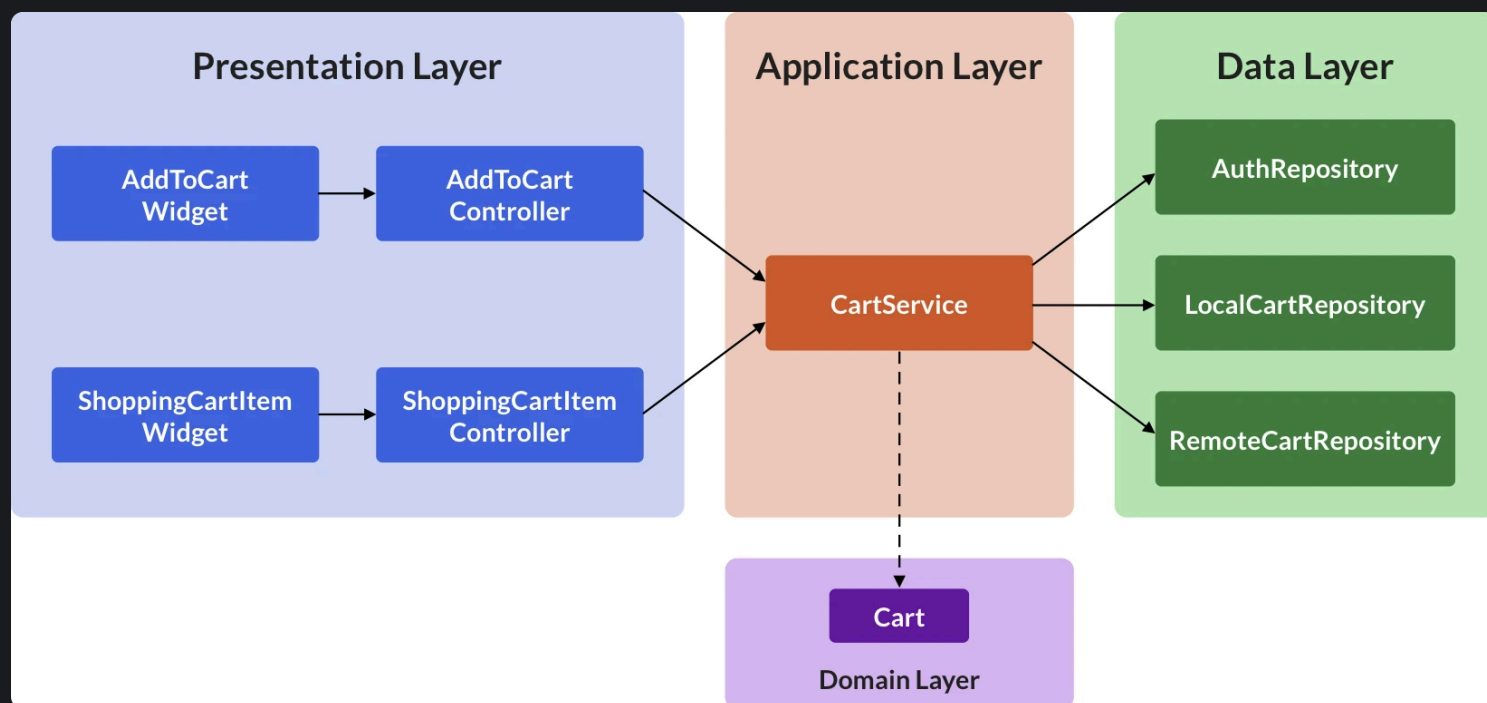
```

final shoppingCartItemControllerProvider =
    StateNotifierProvider<ShoppingCartItemController, AsyncValue<void>>
((ref) {
    return ShoppingCartItemController(
        cartService: ref.watch(cartServiceProvider),
    );
});

```

In this case it's ok to call `ref.watch(cartServiceProvider)` and pass it to the constructor directly because `ShoppingCartItemController` has only one dependency. But if we wanted to pass `ref.read` as a `Reader` argument instead, that would be fine too.

That's it. We've now seen how repositories, services, and controllers can serve as building blocks for building a complex shopping cart feature:



Layers and components used by the shopping cart feature

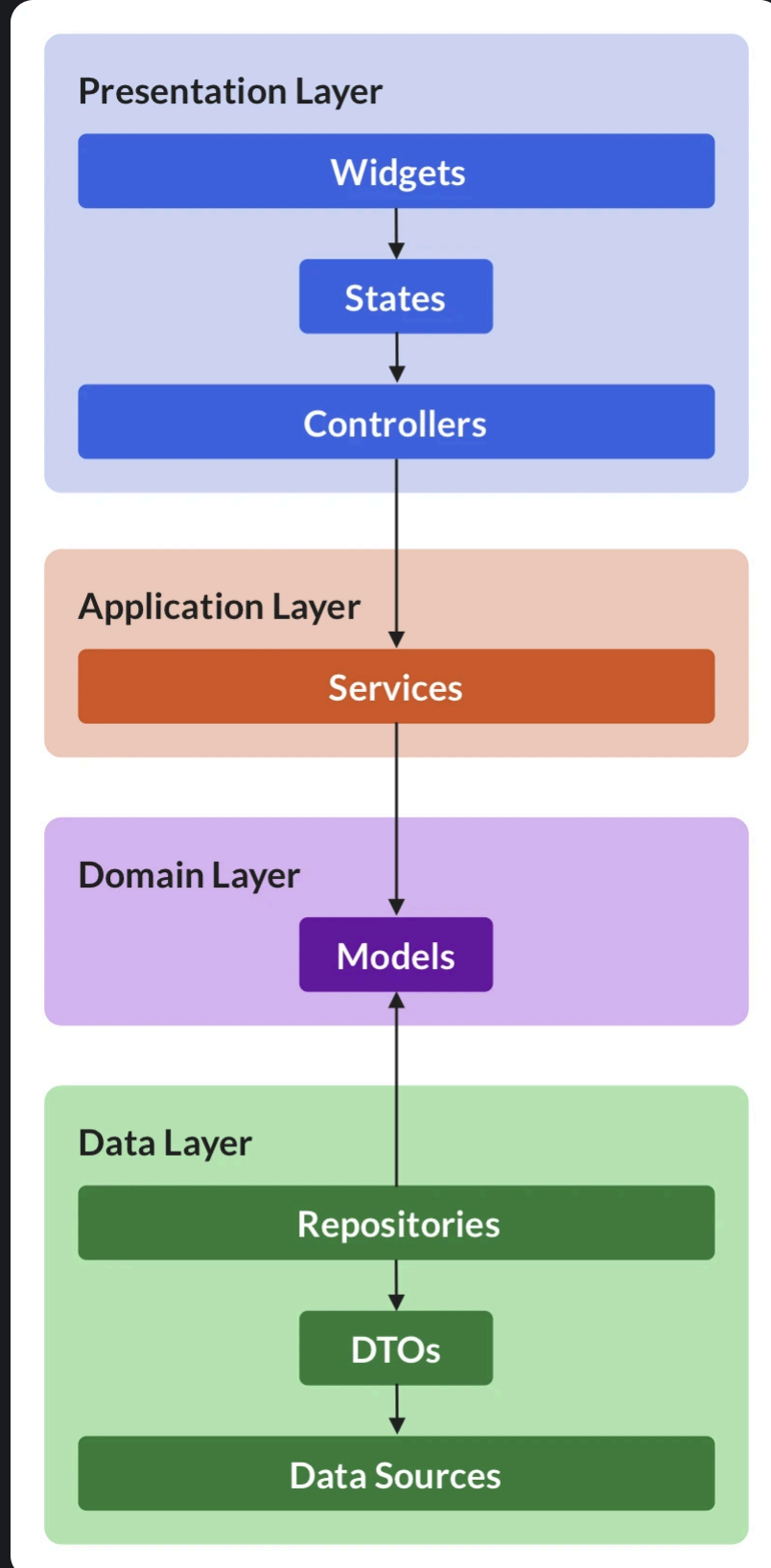
For brevity, I won't show how the widgets or the `AddToCartController` are implemented here, but you can read my article about [Flutter App Architecture: The Presentation Layer](#) to better understand how widgets and controllers interact with each other.

Note about controllers, services, and repositories

Terms such as **controller**, **service**, and **repository** are often confused and used with different meanings in different contexts.

Developers like to argue about these things, and we'll never get everyone to agree on a clear definition of these terms, once and for all. 🙄

The best thing we can do is to **pick a reference architecture** and **use these terms consistently within our team or organization**:



Flutter App Architecture using data, domain, application, and presentation layers. Arrows show the dependencies between layers

Conclusion

We've now completed our overview of the application layer. Since there was a lot to cover, a brief summary is in order.

If you find yourself writing some logic that:

- depends on multiple data sources or repositories

- needs to be used (shared) by more than one widget

Then consider writing a **service class** for it. Unlike controllers that extend `StateNotifier`, service classes don't need to manage any state, as they hold logic that is **not** widget-specific.

Service classes also don't care about data serialization or how to get data from the outside world (that is what the data layer is for).

Lastly, **service classes are often unnecessary**. There's **no point** in creating a service class if all it does is forward method calls from a controller to a repository. In such a case, the controller can depend on the repository and call its methods directly. In other words, the application layer is **optional**.

Finally, if you're following the [feature-first project structure outlined here](#), you should decide if you need service classes **on a feature-by-feature basis**.

Closing notes

App architecture is a deeply fascinating topic, and I've been able to explore it in depth while building a medium-sized eCommerce app (and many other Flutter apps before that).

By sharing these articles, I hope I have helped you navigate this complex topic so that you can design and build **your** own apps with confidence.

And if there is just one thing you should take away from all this, it is that:

Separation of concerns should be a **primary concern** when building apps. Using a **layered architecture** lets you decide what each layer **should** and **should not** do, and establish **clear boundaries** between various components.

Flutter Foundations Course Now Available

I launched a brand new course that covers Flutter app architecture in great depth, along with other important topics like state management, navigation & routing, testing, and much more:



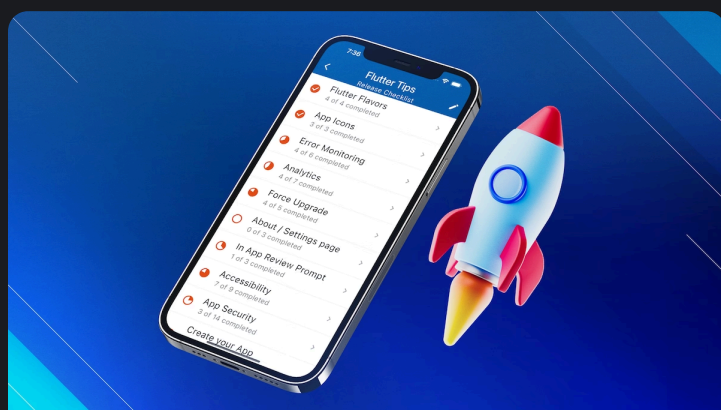
Flutter Foundations Course

INTERMEDIATE TO ADVANCED

Learn about State Management, App Architecture, Navigation, Testing, and much more by building a Flutter eCommerce app on iOS, Android, and web.

Want More?

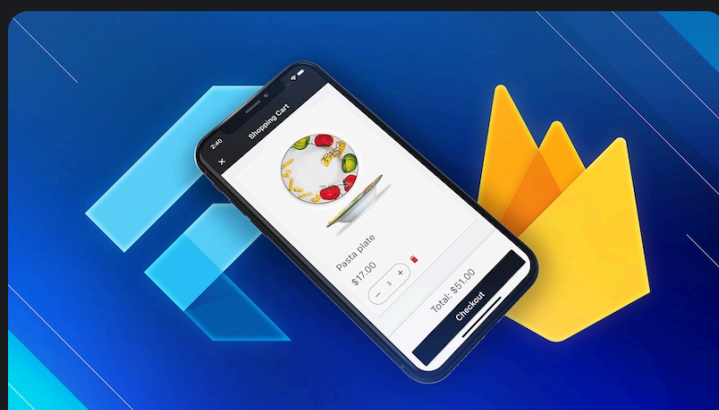
Invest in yourself with my high-quality Flutter courses.



Flutter In Production

INTERMEDIATE TO ADVANCED

Learn about flavors, environments, error monitoring, analytics, release management, CI/CD, and finally ship your Flutter apps to the stores. 🚀



Flutter & Firebase Masterclass

INTERMEDIATE TO ADVANCED

Learn about Firebase Auth, Cloud Firestore, Cloud Functions, Stripe payments, and much more by building a full-stack eCommerce app with Flutter & Firebase.



The Complete Dart Developer Guide

BEGINNER

Learn Dart Programming in depth. Includes: basic to advanced topics, exercises, and projects. Last updated to Dart 2.15.



Flutter Animations Masterclass

INTERMEDIATE

Master Flutter animations and build a completely custom habit tracking application.



CODE WITH ANDREA

Copyright © 2020-present Coding With Flutter Limited

[Contact](#)

[Twitter](#)

[Discord](#)

[GitHub](#)

[RSS](#)

[Meta](#)

[Privacy Policy](#)

20

